

Conectividad Wi-Fi

Pulido Buitrago Nicolas Santiago (7004630) y Martínez
Montaño Bryan Andrey (7004588)

Resumen— En esta práctica se implementó un sistema de ESP32 y el protocolo UDP para la transmisión de datos en una red Wi-Fi local. Se realizó la lectura de dos sensores (temperatura/humedad mediante el sensor DHT11 y nivel de luminosidad mediante un sensor LDR con divisor resistivo), enviando los datos con una frecuencia de superior a 4 Hz hacia una aplicación móvil desarrollada en MIT App Inventor. A su vez, la interfaz móvil permitió el control remoto de cuatro actuadores digitales de tipo ON/OFF (LEDS).

Palabras clave— ESP32, Wi-Fi, UDP, DHT11, LDR, Leds, App móvil.

I. INTRODUCCIÓN

En esta práctica se empleó una tarjeta de desarrollo ESP32 como servidor UDP dentro de una red Wi-Fi local para establecer una comunicación con un dispositivo móvil Android. El objetivo principal consistió en adquirir variables físicas de sensores (DHT11 y LDR) enviándolas periódicamente a una aplicación desarrollada en MIT App Inventor, mientras se recibían comandos de control remoto para activar o desactivar cuatro actuadores digitales.

II. MÉTODOS Y MATERIALES

Para el desarrollo de la práctica se usó un microcontrolador ESP32, un sensor DHT11 y LDR, cuatro Leds y un celular. El DHT11 y el LDR se comunicó con el ESP32, el LDR, mediante divisor resistivo, para obtener las variables de temperatura, humedad e intensidad lumínica. Posteriormente, el ESP32 procesó las lecturas y las transmitió hacia un aplicación móvil, por medio del Wi-Fi, y de la aplicación se transmitieron datos de encendido y apagado a la ESP, por medio del cual se encendían cuatro LEDS.

A. Asignación de pines

La Tabla 1 resume los pines definidos en el programa entregado. Se asignaron un DAC al LDR, uno a lectura del DHT11 y cuatro digitales para los leds.

TABLA I. ASIGNACIÓN DE GPIO EMPLEADA

Elemento	PINES utilizados	Función
LDR	34	Lectura DAC
DHT11	4	Lectura de datos
Leds	12, 13, 14, 27	Salida digital
Wi-Fi	UDP	Transmisión de datos

Fuente: Elaboración propia a partir del código.

B. LÓGICA DE CONTROL

El sistema opera sobre una arquitectura cliente-servidor basada en el protocolo de datagramas de usuario (UDP) en una red Wi-Fi local, la ESP32 realiza la lectura del sensor DHT11 y del canal ADC del LDR a una frecuencia de 4 Hz. Los datos son estructurados en una trama delimitada por comas (formato Temp,Hum,Lum) y enviados mediante datagramas UDP a la dirección IP y puerto específico del dispositivo móvil. El servidor UDP en la ESP32 escucha continuamente los paquetes entrantes enviados desde la App. Al recibir un comando de control, procesa la instrucción e inválida o conmuta el estado lógico del GPIO correspondiente.

Para elegir el protocolo de transporte de datos por internet, se observó el más adecuado entre TCP (Protocolo de Control de Transmisión) y UDP (Protocolo de Datagramas de Usuario), ya que son los dos protocolos de transporte principales.

TCP está orientado a conexión y confiable, antes de enviar cualquier dato, TCP realiza un "saludo de tres vías" para establecer una conexión formal entre emisor y receptor. Durante la transmisión, comprueba que los paquetes lleguen completos, en el orden correcto y sin errores. Si detecta la pérdida de algún paquete, solicita automáticamente su retransmisión, de esta manera garantizando una entrega 100% segura e íntegra de los datos. Aunque esto genera mayor latencia (retardo) y consume más memoria y ancho de banda debido a la supervisión constante, se usa principalmente en páginas web, correo electrónico, transferencia de archivos y servicios de acceso remoto.

UDP es un protocolo ligero y de alta velocidad, envía los paquetes de datos directamente al destinatario sin establecer una conexión previa ni esperar confirmaciones de recepción. Si un paquete se pierde o llega desordenado, no intenta recuperarlo ni reorganizarlo; la aplicación receptora simplemente continúa con el siguiente paquete entrante, tiene una transmisión extremadamente rápida con muy baja latencia y consumo mínimo de recursos del sistema, pero no ofrece garantías de entrega ni de orden en los datos, se usa principalmente en la Transmisión de datos de sensores en tiempo real, streaming de video/audio en vivo, juegos en línea, y llamadas de voz.

¹Documento elaborado el 04 de octubre de 2026.

Se eligió UDP debido a su baja latencia y transmisión en tiempo real, ya que envía los datos directamente sin necesidad de establecer un saludo de manos previo ni confirmar cada paquete entregado, esto permite actualizar las lecturas de los sensores (DHT11 y LDR) de forma constante e inmediata. No se eligió TCP debido a que si un paquete se pierde, bloquea la línea para retransmitirlo, introduciendo retardos o latencias no deseadas. También debido a su menor consumo de recursos en el ESP32.

Para el envío de datos también se observó el Modelo OSI (Sistemas abiertos de interconexión), que es un marco conceptual estandarizado que divide la comunicación entre sistemas en 7 capas jerárquicas: Física: Transmisión de bits por el medio físico (voltaje en cables, radiofrecuencia Wi-Fi), enlace de datos: Direccionamiento físico (direcciones MAC, tramas Wi-Fi/Ethernet), red: Direccionamiento lógico y enrutamiento (direcciones IP), transporte: Comunicación extremo a extremo (End-to-End) entre aplicaciones (TCP / UDP), sesión: Gestión de sesiones entre aplicaciones, presentación: Formato, codificación y encriptación de datos, aplicación: Interfaz final accesible para el usuario/programa (App móvil, HTTP, etc.).

El envío de datos de esta práctica corresponde a la Capa de Transporte (Capa 4 del Modelo OSI), debido a que es la encargada de establecer el canal de comunicación entre dos procesos o aplicaciones que se ejecutan en dispositivos distintos dentro de la red (en este caso, el proceso en la ESP32 y la aplicación en MIT App Inventor en el celular). Pertenecer a esta capa por la identificación por Puertos, que permite dirigir los paquetes a un proceso específico en la red mediante números de puertos origen y destino, el control de Datagramas, que se encarga de encapsular los datos provenientes de la aplicación en 8 bytes (que contiene puertos, longitud del paquete y una verificación de errores) e independencia de la red, ya que la capa de transporte independiza a la aplicación de los detalles físicos e IP de la red Wi-Fi local.

Para el cifrado, se observaron dos tipos de cifrado a nivel de la capa de transporte: Sin cifrado (Texto plano / Unencrypted), y a nivel de capa de Física (Red Wi-Fi local): Cifrado WPA2-PSK / WPA 3-AES.1, para la comunicación UDP, se usa ninguno. Las lecturas del DHT11 y LDR, así como los comandos para los Leds, se envían en formato de texto plano estructurado. Para la red Wi-Fi: Se delega la seguridad en la capa de acceso a la red Wi-Fi utilizando WPA2-Personal (AES) o WPA3, solicitando la clave de la red local al conectar el ESP32. No hay cifrado, debido a que se requiere maximizar la velocidad y minimizar la latencia, si se implementara cifrado simétrico/asimétrico (como TLS/DTLS o AES) requiere que la ESP32 ejecute cálculos matemáticos complejos para encriptar y desencriptar cada paquete en tiempo real. Al enviar los datos por UDP permite cumplir la velocidad que se

quiere. También debido a que MIT App Inventor no posee de forma nativa módulos avanzados de desencriptación. Recibirlo de esta manera facilita la decodificación directa en los bloques de la app. Al operar en una red Wi-Fi local privada, la propia red ya aplica un cifrado físico a nivel de radiofrecuencia (WPA2/WPA3), lo que evita que personas fuera de la red Wi-Fi puedan interceptar las ondas de radio en texto claro.

La aplicación Móvil (MIT App Inventor) en la interfaz gráfica de la aplicación móvil cuenta con los siguientes componentes: Indicadores numéricos y visuales que muestran en tiempo real los valores actualizados de temperatura (°C), humedad (%) y luminosidad (%), interruptores digitales para el control independiente de los cuatro leds.

III. RESULTADOS

Se obtuvo una comunicación estable sobre la red Wi-Fi local con baja latencia y sin pérdida apreciable de paquetes UDP. Las lecturas del DHT11 y LDR se actualizaron fluidamente en la aplicación Android. Así mismo, los cuatro actuadores respondieron de manera inmediata a los comandos enviados desde la interfaz móvil. El código fuente (para la ESP32 y MIT App Inventor), esquemáticos de conexión y el video de demostración del funcionamiento se encuentran en el siguiente repositorio

<https://github.com/Nicolas-Pulido/Laboratorio-2>

IV. CONCLUSIONES

- Se diseñó e implementó exitosamente una aplicación utilizando la ESP32 y el protocolo UDP para el monitoreo de sensores y control de actuadores vía Wi-Fi.
- El protocolo UDP demostró ser eficiente para la transmisión en tiempo real de variables físicas, manteniendo una baja latencia en la red local.
- La integración con MIT App Inventor permitió crear una interfaz de usuario para la supervisión y control del sistema..

V. REFERENCIAS

- [1] J. V. S. Morais, ESP32 con IDF: La Guía Profesional. Barcelona, España: Marcombo, 2023.
- [2] V. O. Öner, Developing IoT Projects with ESP32, 2nd ed. Birmingham, U.K.: Packt Publishing, 2023.
- [3] A. Ibrahim, Hands-On ESP32 with Arduino IDE: Practical Guide to IoT Development. Birmingham, U.K.: Packt Publishing, 2023.
- [4] L. D. Medley, Mastering ESP32 Systems: Designing, Programming, and Deploying Smart Embedded Applications. New York, NY, USA: Apress, 2024.
- [5] Universidad Militar Nueva Granada, Guía de Práctica de Laboratorio: Conectividad WIFI, Asignatura Laboratorio de Micros, 2026.



Laboratorio micros, Quinto semestre
Universidad Militar Nueva Granada, Cajicá, D.C.
Ingeniería en Mecatrónica

Repositorio:

<https://github.com/BryanM28-git/Laboratorio-WiFi-.git>